CPU Scheduling Studio

Test and Correction Report

Calculation accuracy · comparison modes · interface regression

---

Prepared 19 October 2026 · Manus AI

Contents

1 Project overview 1
2 Test method and coverage 1
3 Scheduling calculation results 2
4 Modes, inputs, and regression results 2
5 Defect fixed 3
6 Running the project and tests 3
7 Remaining limitations 3

1 Project overview

The supplied ZIP contains a responsive, dependency-free browser simulator named Schedulab. The JavaScript scheduler implements six algorithms ported from the supplied C source: FCFS, non-preemptive SJF, preemptive SRTF, Round Robin, non-preemptive Priority, and the Round Robin + Priority hybrid. The interface provides single-algorithm runs, two-algorithm comparisons, and custom comparisons of one to six algorithms.

The project structure and visual design were preserved. The scheduling engine required no correctness changes. One mobile layout defect was found and fixed: at a 375-pixel viewport, the page originally had a 445-pixel document scroll width because the visually hidden “Delete” table heading extended beyond the screen. A small CSS adjustment anchored and clipped that accessibility label; after the change, the 375-pixel page width is 375 pixels.

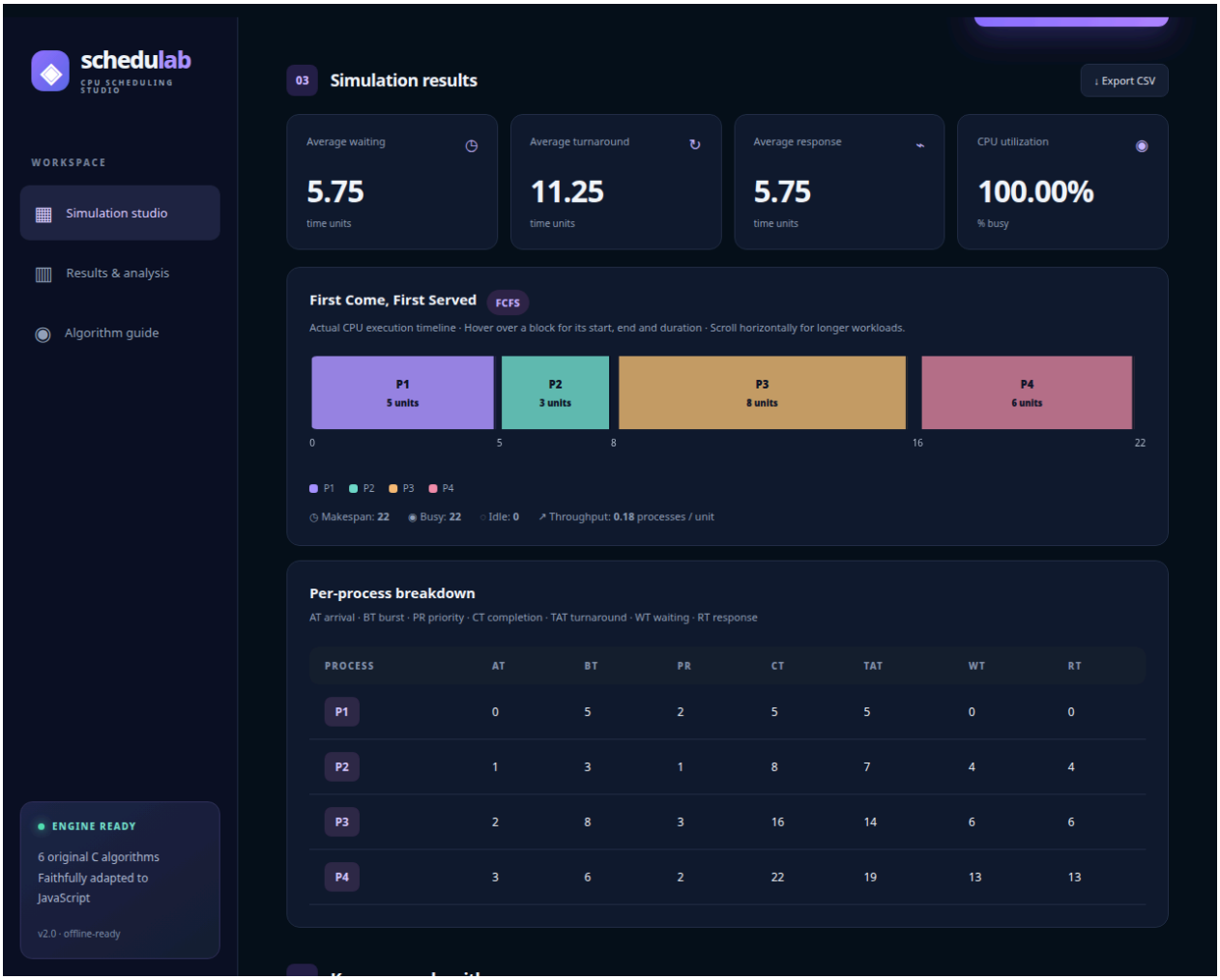


图 1 Actual Chromium capture of the default FCFS run, including metrics, Gantt chart, and per-process results.

2 Test method and coverage

The project’s Node.js suite was run with Node 22.13.0. All 15 tests passed. This includes the original reference tests and a new deterministic oracle suite. The independent oracle ran 320 seeded workloads, with 1–8 processes each, across all six algorithms and quantum values 1, 2, 4, and 9: 7,680 full simulations. Each run compared the execution bars, completion/turnaround/waiting/response times, averages, CPU utilization, idle time, and throughput. A fixed edge test also covers simultaneous arrivals, equal priority, an idle gap, and quantum 500.

The browser interface was served over local HTTP and tested in Chromium. All 60 UI checks passed. The checks covered each algorithm as an independent run, the three operating modes, Gantt charts and tables, quantum controls, process add/remove and the 30-process cap, CSV export, theme switching, invalid values, local asset paths, and runtime errors. Responsive overflow was checked at 320, 360, 375, 600, 601, 768, 860, 861, 1,024, 1,250, 1,280, and 1,440 pixels. No uncaught JavaScript errors or failed network requests remained.

3 Scheduling calculation results

The sample workload used for the reference comparison was P1 (AT 0, BT 5, PR 2), P2 (AT 1, BT 3, PR 1), P3 (AT 2, BT 8, PR 3), and P4 (AT 3, BT 6, PR 2). Expected values were calculated with a separate reference simulator using the same documented tie and queue rules. The observed values below matched those expected values in every case.

Algorithm	Observed Gantt order, quantum 2	Avg WT / TAT / RT	CPU
FCFS	P1 → P2 → P3 → P4	5.75 / 11.25 / 5.75	100%
SJF	P1 → P2 → P4 → P3	5.25 / 10.75 / 5.25	100%
SRTF	P1 → P2 → P1 → P4 → P3	5.00 / 10.50 / 4.25	100%
Round Robin	P1 → P2 → P3 → P1 → P4 → P2 → P3 → P1 → P4 → P3 → P4 → P3	9.75 / 15.25 / 2.00	100%
Priority	P1 → P2 → P4 → P3	5.25 / 10.75 / 5.25	100%
RR + Priority	P1 → P2 → P2 → P1 → P4 → P1 → P4 → P4 → P3 → P3 → P3 → P3	5.75 / 11.25 / 4.25	100%

For the FCFS sample, the expected and actual per-process values were identical:

Process	AT	BT	CT	TAT	WT	RT
P1	0	5	5	5	0	0
P2	1	3	8	7	4	4
P3	2	8	16	14	6	6
P4	3	6	22	19	13	13

The FCFS averages are 5.75 time units waiting, 11.25 turnaround, and 5.75 response. Total burst is 22 and makespan is 22, so CPU utilization is 100%, idle time is 0, and throughput is 4/22 = 0.1818 processes per time unit.

For an SRTF workload with P1 (AT 3, BT 8) and P2 (AT 5, BT 2), the expected and actual Gantt order was IDLE 0–3, P1 3–5, P2 5–7, P1 7–13. The observed metrics were P1 (CT 13, TAT 10, WT 2, RT 0) and P2 (CT 7, TAT 2, WT 0, RT 0). The initial three idle units were included in utilization: 10/13 = 76.92%.

Round Robin quantum variation was verified against the same sample input. The observed number of Gantt slices matched the independent expectation: 22 at quantum 1, 12 at quantum 2, 7 at quantum 4, and 4 at quantum 9. The browser test also confirmed that quantum 1 and quantum 4 produce different rendered execution segmentation.

4 Modes, inputs, and regression results

Single mode was run independently for all six algorithms. Compare-two mode selected exactly two algorithms and rendered two separate runs and two comparison rows. Compare-all mode selected all six algorithms and rendered six runs, six comparison rows, and 24 metric-chart bars. Turning all algorithms off produced a visible validation message rather than a silent or stale result.

Input validation rejected a zero burst, a negative arrival time, a fractional burst, a zero quantum, and an empty workload. Add and remove actions worked, including adding after Clear. The Add control disabled

at 30 processes and re-enabled after removing a row. CSV export produced a downloadable, correctly quoted CSV with the expected header and result values. Sample reset, theme toggling, and quantum increment/decrement controls also worked.

Every referenced local stylesheet and JavaScript module returned HTTP 200. A deliberate request to a nonexistent asset returned HTTP 404 as expected. The algorithm picker is a group of buttons rather than a native dropdown; no dropdown control is present in the supplied interface.

5 Defect fixed

The sole confirmed defect was mobile horizontal overflow caused by the offscreen accessibility header in the process-entry table. Before correction, a 375-pixel viewport reported a document width of 445 pixels. After the CSS change, the document width matched the viewport at 375 pixels. Regression checks then passed at all 12 measured widths from 320 to 1,440 pixels. No scheduling calculation defects were found in the six implementations during the oracle and reference-case tests.

6 Running the project and tests

Extract the corrected ZIP. From the `CPU-Scheduling-Studio` directory, start a local server:

```
python3 -m http.server 8000
```

Open `http://localhost:8000` in a modern browser. A local server is recommended because some browsers restrict JavaScript modules on `file://` URLs. To run the automated scheduling tests, use Node.js 18 or newer from the same project directory:

```
node --test tests/*.test.mjs
```

The app itself needs no Node.js installation, package manager, CDN, or network access.

7 Remaining limitations

Browser interaction and responsive behavior were tested in Chromium on Linux, not in Firefox, Edge, Safari, a physical phone, or a screen reader. Direct `file://` startup was not tested; the included README explains how to use the local HTTP server if a browser blocks ES modules. The original C executable was not compiled or run because the supplied source depends on `graphics.h`/WinBGIm; the deliverable tested here is the JavaScript browser app, and the original C file remains preserved in the ZIP.

Inputs are integers with the documented limits of 1–30 processes, arrival 0–10,000, burst 1–500, priority 0–999, and quantum 1–500. CPU utilization and throughput measure elapsed time from t=0, including initial idle time. RR + Priority follows the supplied custom hybrid policy: it selects the highest-priority-number-ranked ready task at each quantum boundary and does not preempt mid-quantum on arrival. Equal-priority ties follow input or queue order as documented in the project.